UNIVERSIDAD NACIONAL DE LOJA

CARRERA DE COMPUTACIÓN



Asignatura: Estructura de Datos

Informe Técnico

Pruebas unitarias de navegación bidireccional e integración en repositorio

Grupo N.°: 9

Integrantes:

Isaac Vire
Danny Beltran
Noelia Bustan
Lucas Orellana
Axel Jimenez

Docente:

PhD(c). Cristian R. Narváez Guillén, Mg. Sc.

Loja, 20/10/2026

1. Introducción y Objetivos del Avance

En el marco del Aprendizaje Basado en Problemas (ABP) para la asignatura, el presente avance contempla la transición de estructuras lineales simples hacia una Lista Doblemente Enlazada con ListaDobleNavegacion. Esta implementación busca garantizar la navegación bidireccional con complejidad temporal O o 1 y la liberación explícita de memoria para el recolector de basura o también conocido como Garbage Collector.

1.1. Objetivos Específicos

- Programar los componentes de nodos y listas con doble enlace sin depender de colecciones nativas de JavaScript.
- Garantizar la liberación de referencias desvinculadas para optimizar la gestión de memoria en el Memory Heap.
- Implementar y validar la solución mediante pruebas unitarias automatizadas con la librería nativa nodtest.

2. Enlace al Repositorio y Evidencia Git

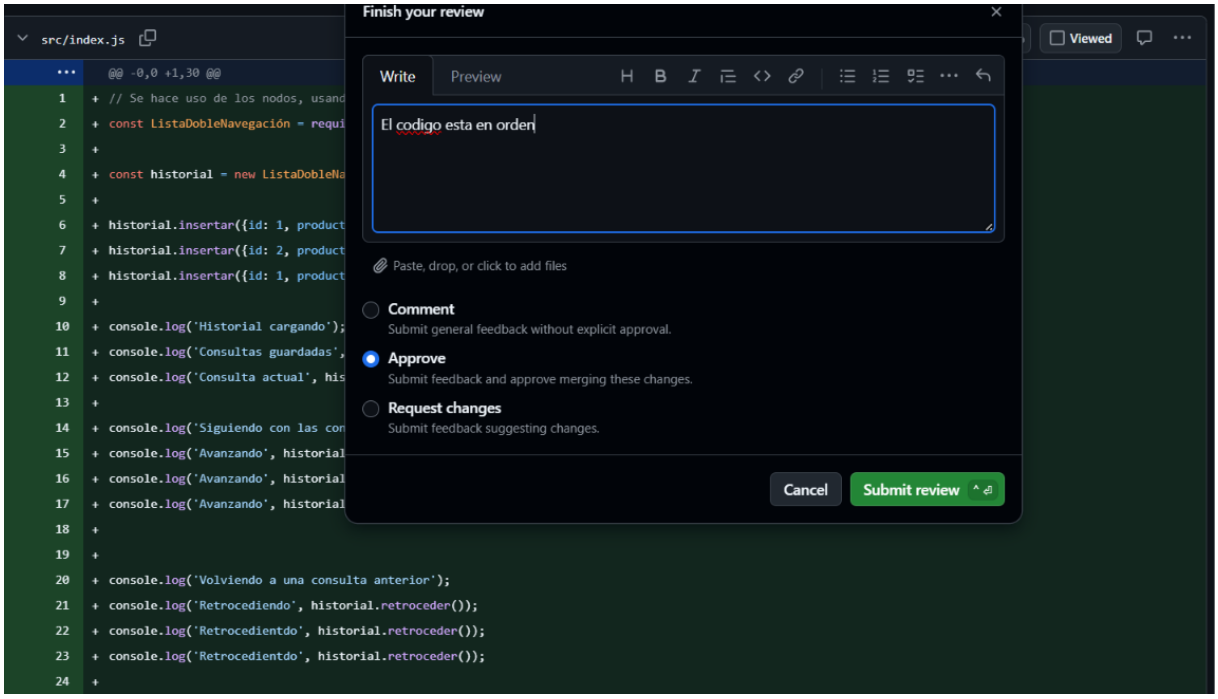
2.1. URL del repositorio

https://github.com/B3kk4r/Estructura-de-datos-Grupo9.git

2.2. Rama de trabajo

Se trabajo en la Rama: feature/AA4-lista-doble-grupo09

2.3. Captura del Pull Request aprobado



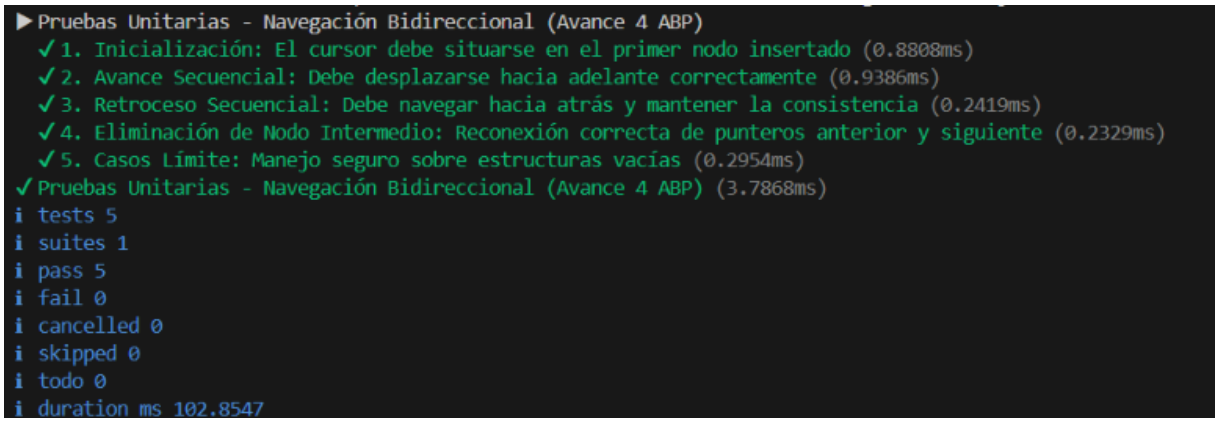
2.4. Commit Hash del merge a la rama principal

`https://github.com/B3kk4r/Estructura-de-datos-Grupo9/commit/87c747b2f409fbbb13f7620be9f27cf12b1df422`

3. Cobertura de Pruebas

Una vez todo esta correctamente configurado y ubicado, se pone node –test tests/navegacion.test.js, para realizar las pruebas unitarias automatizadas.

3.1. Captura de la consola



3.2. Tabla de resultados

N.°	Prueba	Método(s) evaluado(s)	Resultado
1	Inicialización	obtenerActual() y insertar()	ok
2	Avance secuencial	avanzar()	ok
3	Retroceso secuencial	retroceder() y avanzar()	ok
4	Eliminación de nodo intermedio	avanzar(), eliminarActual(), obtenerActual() y retroceder()	ok
5	Casos límite (lista vacía)	avanzar(), eliminarActual(), obtenerActual() y retroceder()	ok

Cuadro 1: Cobertura de pruebas unitarias

4. Análisis de Complejidad Temporal y Espacial

Método	Tiempo	Espacio	Justificación
`avanzar()`	O(1)	O(1)	Con este método se puede verificar que tanto el nodo actual como el siguiente existan, asi como avanzar al siguiente nodo
`retroceder()`	O(1)	O(1)	Con retroceder se hace lo mismo que con avanzar, se verifica que tanto el nodo actual como el anterior existan y se reasigna el cursor al nodo anterior
`eliminarActual()`	O(1)	O(1)	Con este método lo que se busca es eliminar de forma segura la referencia a un nodo tanto por parte del nodo anterior como por parte del siguiente, así como actualizar cabeza, colo y actual para que la navegación entre nodos no se vea afectada

Cuadro 2: Complejidad de los métodos de `ListaDobleNavegacion`

Los métodos avanzar(), retroceder() y eliminarActual() tienen complejidad O(1) en tiempo y espacio porque trabajan directamente sobre el nodo apuntado por el cursor y sus vecinos, sin recorrer la lista. A diferencia de un arreglo, donde eliminar un elemento intermedio obliga a desplazar los elementos posteriores, en la lista doble solo se reconecta un número fijo de punteros. Asimismo, a diferencia de una lista simplemente enlazada, el enlace anterior permite retroceder o eliminar sin recorrer la lista desde la cabeza. Solo se emplean variables locales de tamaño constante, como nodoAEliminar.

4.1. Liberación de referencias y Garbage Collector

En eliminarActual(), tras reconectar los nodos vecinos, se asigna null a las referencias siguiente y anterior del nodo eliminado. De esta forma el nodo queda desvinculado de la lista y ninguna parte activa del programa lo referencia, por lo que el Garbage Collector de JavaScript puede liberar su memoria del Memory Heap sin complicaciones. Esta limpieza explícita evita que queden enlaces residuales que retengan el nodo en memoria.

5. Conclusión

En este avance se implementó y probó la estructura ListaDobleNavegacion, que permite desplazarse hacia adelante y hacia atrás y eliminar el nodo activo con complejidad O(1), sin recorrer la lista. Las 5 pruebas unitarias se ejecutaron con éxito, y la estructura se integró al proyecto de precios agrícolas como historial de consultas. Además, la liberación explícita de referencias en eliminarActual() facilita el trabajo del Garbage Collector. El trabajo en ramas, Pull Requests y merge a main permitió una integración colaborativa y verificable, con trazabilidad completa del aporte de cada integrante.

6. Declaración de Ética e Inteligencia Artificial

6.1. Uso de asistentes de IA

Se usó la IA para solventar dudas respecto al código dado y poder entenderlo de mejor manera, así como ser capaz de verificar si las declaraciones hechas respecto a Big-O o a la utilidad de las listas doblemente enlazadas eran correctas y acertadas.

6.2. Herramienta(s) utilizada(s)

Perplexity y Claude

6.3. Prompts empleados

De manera detallada pero sencilla, como un profesor de programación, indicame lo que hace el código que te presente a continuación:

En base a los conceptos que rodean a Big-O y a las listas doblemente enlazadas, que tan acertado es el párrafo redactado:

6.4. Declaración de autoría técnica

Los integrantes del Grupo N.9 declaramos que la implementación y la verificación de la solución técnica son de nuestra autoría. La estructura de datos ListaDobleNavegacion, la integración en el proyecto ABP, las pruebas unitarias y trabajo en GitHub fueron desarrollados, ejecutados y revisados por el equipo, y cada integrante comprende el código que aportó.

El asistente de inteligencia artificial como perplexity y claude se utilizó únicamente como apoyo para la investigación, sobre consulta de conceptos sobre listas doblemente enlazadas, notación Big-O y la forma de trabajar en GitHub, y para la corrección de este informe. Todo el contenido generado fue revisado, contrastado y editado por los integrantes, quienes asumimos la responsabilidad total del trabajo presentado.